TECHNICAL UNIVERSITY OF MOLDOVA

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automatics



LABORATORY WORK NO. 1

Concurrency in Practice: Multithreading, Shared State and I/O Performance

Course: Network Programming

Author:

Oberst Eduard

Student of group: FAF-243

Verified by:

Cristofor Fistic

Chișinău — 2026

Contents

1 Introduction and Objectives 3

2 Hardware and Software Environment 3

2.1 Architectural Implications....4

2.2 Measurement Methodology....4

3 Theoretical Background 5

3.1 Concurrency and Parallelism....5

3.2 Speed-up, Efficiency and Amdahl's Law....5

3.3 Shared State, Race Conditions and Synchronisation....6

3.4 Producer–Consumer Pattern and Bounded Queues....6

3.5 CPU-Bound and I/O-Bound Work....6

4 Implementation 7

4.1 Dataset....7

4.2 Task 1a: Splitting One File Among Threads....7

4.3 Task 1b: Three Ways to Count....8

4.4 Task 1c: One Reader, N Workers, Bounded Queue....8

4.5 Task 2: Many Files on Slow Storage....8

5 Experimental Results 9

5.1 Task 1a: Counting with Threads....9

5.2 Task 1b: Three Ways to Count....11

5.3 Task 1c: Producer and Consumers....13

5.4 Task 2: Many Files on Slow Storage....14

5.4.1 USB Drive with Replug....14

5.4.2 USB Drive without Replug....15

5.4.3 A Note on the First Attempt....17

5.4.4 Reference: Internal NVMe SSD....17

6 Task Questions 18

6.1 Task 1 Questions....18

6.1.1 Question 1: At which number of threads does the speed-up stop growing? Relate it to physical cores and logical processors....18

6.1.2 Question 2: Why do 32 and 64 threads not help? Are they slower, and why?....18

6.1.3 Question 3: Does the peak memory depend on the number of threads? Why?....19

6.1.4 Question 4: Rank the three versions of Task 1b by time. Which of them

are correct? Explain each result....19
6.1.5 Question 5: Compare Task 1c with Task 1a. Which one is faster, and why? At which number of workers does Task 1c stop getting faster?. .19
6.1.6 Question 6: What would happen with batches of a single line? And with a queue that has no size limit?....20
6.2 Task 2 Questions....21
6.2.1 Question 1: Compare the time on one thread with Task 1a on one thread. On the USB drive, is the work CPU-bound or I/O-bound?....21
6.2.2 Question 2: On the USB drive, do more threads help? Why not, when lecture 1 showed that I/O-bound work benefits from concurrency?.....21

7 Control Questions 22

7.1 What is the difference between a physical core and a logical processor? How did it show in your charts?....22
7.2 How did you make sure that no line was counted twice or skipped at the boundary between two parts of the file?....22
7.3 Explain step by step how unsynchronised shared counters lose updates.....22
7.4 Why are per-thread counters faster than shared counters protected by a lock? 23
7.5 What does "I/O-bound" mean for a USB drive, and for a server waiting on the network? Why does concurrency help in only one of the two cases?....23
7.6 What is the difference between dividing work into equal parts in advance and taking it from a queue? When is each one better?....23

8 Conclusions 24

9 Source Code and Build Instructions 25

9.1 Build....25
9.2 Generating the Data....25
9.3 Running One Configuration....25
9.4 Reproducing the Measurements....25

1 Introduction and Objectives

Modern processors have many cores, and a program uses them only if it divides its work among several threads. Whether this actually makes the program faster depends on the hardware and on how the threads share data and devices. This laboratory work measures it on a real machine with one simple job: counting the negative, positive and zero numbers among 50 000 000 integers stored in a text file.

The objectives of the laboratory work are:

1. **Thread scalability (Task 1a):** split one file among 1, 2, 4, 8, 16, 32 and 64 threads, measure the time, the speed-up and the peak memory, and relate the point where the speed-up stops growing to the number of physical cores and logical processors.
2. **Shared state and synchronisation (Task 1b):** count with unsynchronised shared counters, with shared counters protected by a lock and with per-thread counters; check which versions are correct and explain what each one costs.
3. **Producer–consumer pipeline (Task 1c):** connect one reader thread and N worker threads with a bounded queue, compare the result with Task 1a, and study the effect of the batch size and of the queue capacity.
4. **CPU-bound and I/O-bound work (Task 2):** process 8 files stored on a USB flash drive with the OS file cache bypassed, compare with the internal SSD, and explain when more threads help an I/O-bound program.

The program is written in **C# 14 / .NET 10**. The source code, the build instructions and the commands used for every measurement are given in Section 9. The raw measurements are in `results/*.csv`, the full program output in `results/logs/` and the charts in `results/charts/`.

2 Hardware and Software Environment

All measurements were made directly on the physical machine described in Table 1 (no virtual machine).

Table 1: Hardware and software environment

Component	**Specification**
CPU	13th Gen Intel Core i5-13450HX (hybrid: 6 Performance-cores at up to 4.6 GHz with Hyper-Threading + 4 Efficient-cores at up to 3.4 GHz, 20 MB L3)
Physical cores / logical processors	**10 physical cores** (6 P + 4 E) / **16 logical processors** ($6 \times 2 + 4 \times 1$)
RAM	14.8 GiB (as reported by the OS)
Internal disk	Solidigm SSDPFINW010TZL, NVMe SSD
USB flash drive	Generic "Flash Disk" (Alcor Micro controller, USB ID 058f:6387), 16 GB (14.8 GiB), FAT32. It is a **USB 2.0** device, so the link runs at USB 2.0 High Speed, **480 Mbit/s**. Measured sequential read **≈ 13.6 MB/s**
Operating system	Ubuntu 26.04.1 LTS, Linux 7.0.0-38-generic, x86-64
Language / runtime	C# 14, .NET SDK 10.0.114, runtime .NET 10.0.14; workstation non-concurrent GC; tiered compilation off (methods are fully optimised from the first call)
Power	on AC power, power profile **performance**

2.1 Architectural Implications

The i5-13450HX is a *hybrid* processor: its 16 logical processors are not 16 equal cores. This determines what the scaling curves can look like:

- **Up to 6 threads**, every thread can run alone on a fast P-core, so an almost linear speed-up is expected.
- **From 7 to 16 threads**, the additional threads run either on the 4 E-cores, which are slower (3.4 GHz instead of 4.6 GHz, simpler cores), or as the second hardware thread (Hyper-Threading) of a P-core, sharing that core's execution units and caches with the first thread. Each of them adds much less than a whole P-core.
- **Beyond 16 threads** there is no free hardware left: the extra threads only take turns on the same 16 logical processors.
- The data file (369.4 MB) fits easily in RAM, so after a warm-up run Task 1 reads it from the OS page cache, not from the SSD.

2.2 Measurement Methodology

- **Data:** `numbers.txt`, 50 000 000 random integers in [−1 000 000, 1 000 000], 369.4 MB, generated with a fixed seed (2026). True totals: **negative 25 000 383, positive 24 999 592, zero 25, total 50 000 000**. Every task compares its totals with these and prints OK/WRONG (column `correct` in the CSV files).

- **Threads** are OS threads created with `new Thread(...)`; the work is split by the program itself (no thread pool, no `Parallel.For`).
- **Time** = wall-clock time (`Stopwatch`) from creating the threads until all of them have finished and the totals are added up.
- **Peak memory** = peak resident set size of the process (`VmHWM` in `/proc/self/status`). Every configuration runs in a **fresh process**, so the peak belongs to that configuration only. About 30 MiB of it is the .NET runtime itself. The program also prints the total bytes allocated on the managed heap (`GC.GetTotalAllocatedBytes`).
- **Repetitions:** every configuration was run 5 times (Task 1a) or 3 times (Task 1b sweep, Task 1c), the runs of all thread counts interleaved; tables and charts show the **median**. A warm-up run before each series loads the file into the OS page cache, so in Task 1 the file is read from RAM, not from the SSD (the SSD alone would add ≈0.1–0.2 s, see Section 5.4.4).

3 Theoretical Background

3.1 Concurrency and Parallelism

Concurrency means that several tasks are in progress during the same period of time; on one processor the scheduler interleaves them. *Parallelism* means that tasks execute at the same instant on different hardware. Threads always give concurrency, but they run in parallel only up to the number of logical processors (16 on this machine), and at full speed only while each of them has a physical core of its own.

3.2 Speed-up, Efficiency and Amdahl's Law

If T_1 is the time with one thread and T_p the time with p threads, the speed-up and the parallel efficiency are

$$S(p)=\frac{T_1}{T_p}, \qquad E(p)=\frac{S(p)}{p} \tag{1}$$

If only a fraction f of the work can be done in parallel and the rest, 1 − f, is sequential (starting the threads, adding up the totals, reading the input on a single thread), Amdahl's law limits the speed-up:

$$S(p)=\frac{1}{(1-f)+\frac{f}{p}} \leq \frac{1}{1-f} \tag{2}$$

Even with infinitely many threads the time cannot fall below the time of the sequential part. In practice the speed-up is lower still, because extra threads bring costs of their own (creating them, scheduling them, competing for shared resources).

3.3 Shared State, Race Conditions and Synchronisation

A *data race* happens when two threads access the same memory location at the same time, at least one of them writes, and nothing orders the accesses. The statement `counter++` is not one indivisible operation but three steps:

1. **Load** the value from memory into a register;
2. **Add** 1 to the register;
3. **Store** the register back to memory.

If two threads interleave these steps, one store overwrites the other and an increment is lost (a *lost update*). The usual remedies are:

- **Mutual exclusion** (`lock` / `Monitor`): only one thread at a time executes the critical section. It is correct, but the threads are serialised, every acquire and release costs atomic operations, and a waiting thread may be put to sleep and woken up by the OS.
- **Atomic instructions** (`Interlocked.Increment`, `lock xadd` on x86): each increment is indivisible without a lock, but all threads still write to the same memory.
- **No sharing:** each thread counts in private variables and the results are added once at the end (a *reduction*). There is nothing to synchronise while counting.

Shared data also has a hardware cost: caches work with 64-byte *cache lines*, and a line written by several cores has to move from core to core on every write (cache-coherence traffic), even when the accesses are correctly synchronised.

3.4 Producer–Consumer Pattern and Bounded Queues

In the producer–consumer pattern, producers put work items into a queue and consumers take them out. A *bounded* queue of capacity C makes the producer wait when C items are already waiting (**back-pressure**, which keeps the memory bounded) and makes the consumers wait when it is empty. Every `Add` and `Take` is a synchronised operation, so items are grouped into **batches** to spread this cost over many lines. Unlike a split made in advance, a queue balances the load automatically: a free consumer simply takes the next item.

3.5 CPU-Bound and I/O-Bound Work

A program is *CPU-bound* when its time goes on computation; more cores make it faster. It is *I/O-bound* when most of its time is spent waiting for a device or for the network. Concur-

rency helps I/O-bound work only when the waits are **independent** and can overlap (for example many network requests, each waiting for a different server); it does not help when all threads queue for **one device that is already at its bandwidth limit**.

4 Implementation

The whole laboratory work is one console program, `Lab1`; the task and its parameters are chosen with command-line arguments (Section 9). Threads are always real OS threads created with `new Thread(...)`. Table 2 lists the source files.

Table 2: Structure of the source code

File	Contents
`Program.cs`	command dispatch and `help` text
`Args.cs`	command-line parser (`--key value`, `--flag`)
`Numbers.cs`	line parser, `Counts`, splitting a file into line-aligned byte ranges
`Measure.cs`	peak memory (`VmHWM`), the expected totals, the machine-readable result line
`DataTools.cs`	`gen` (data file), `split` (8 parts), `info` (hardware)
`Task1a.cs`	Task 1a – each thread reads and parses its own part of the file
`Task1b.cs`	Task 1b – unsynchronised / lock / per-thread counters (+ `Interlocked` as an extra)
`Task1c.cs`	Task 1c – one reader, N workers, bounded `BlockingCollection`
`Task2.cs`	Task 2 – one file = one unit of work, `File.ReadAllBytes`, files split among threads
`Bench.cs`	runs a task for several thread counts, each run in a fresh process, writes CSV
`Charts.cs`	turns the CSV files into SVG charts (time, speed-up, peak memory) and a table

4.1 Dataset

`lab1 gen` writes `data/numbers.txt` (50 000 000 integers, one per line, see Section 2.2) and `data/numbers.txt.counts` with the true totals. For Task 2, `lab1 split` divides it, in order, into `part_1.txt … part_8.txt` (6 250 000 lines, 46.2 MB each) plus `expected.counts`. Every run checks that

$$N_{\text{negative}} + N_{\text{positive}} + N_{\text{zero}} = 50\,000\,000 \tag{3}$$

and compares each of the three totals with the expected value.

4.2 Task 1a: Splitting One File Among Threads

Each thread opens the file itself and processes only its own part of it:

1. Thread i of N gets the raw byte range [size·i/N, size·(i+1)/N).

2. Both ends of the range are moved forward to the next line start (`Numbers.AlignToLineStart`), so a line belongs to the thread whose range contains its first byte (details in Section 7.2).
3. The thread reads its range in 256 KiB chunks with positional reads (`RandomAccess.Read`), parses the bytes directly and counts into local variables.
4. The main thread waits for all threads (`Join`) and adds up the per-thread totals.

4.3 Task 1b: Three Ways to Count

The numbers are loaded into an `int[]` first (0.7 s, not measured). Then 16 threads (= logical processors) each count 3 125 000 numbers, in one of these versions:

1. **Shared, no synchronisation:** all threads increment the same three shared counters. This version is run 3 times, to show that its result changes from run to run.
2. **Shared + lock:** every increment of a shared counter is done inside `lock (gate) { … }`.
3. **Per-thread counters:** each thread counts into its own local variables and the main thread adds up the per-thread totals after `Join`.
4. **Extra − shared + `Interlocked`:** `Interlocked.Increment` on the shared counters (atomic instructions, no lock). It is not required and was added for comparison.

4.4 Task 1c: One Reader, N Workers, Bounded Queue

One reader thread reads the file with `StreamReader.ReadLine()`, puts the lines into `string[10 000]` batches and adds them to a `BlockingCollection<string[]>` with capacity 100 (a bounded blocking FIFO queue). When the file ends it calls `CompleteAdding()` (closes the queue). N worker threads take batches until the queue is closed and empty, parse the lines and count into local variables. The program also records how long the reader waited on a full queue, how long the workers waited on an empty queue, and the longest the queue ever got. The batch size and the capacity are parameters (`--batch`, `--capacity`, where 0 means unbounded), used for the experiments of question 6.

4.5 Task 2: Many Files on Slow Storage

`numbers.txt` was split, in order, into `part_1.txt … part_8.txt` (6 250 000 lines, 46.2 MB each). A file is the unit of work: a thread reads it with **one `File.ReadAllBytes` call into one `byte[]`**, then parses the buffer and counts. The files are divided into equal contiguous groups (1 thread: 8 files, 2 threads: 4 + 4, 4 threads: 2 + 2 + 2 + 2). For every file the program prints how long the read and the parse took.

The 8 files were copied to the USB stick and read only from there. Three series were measured:

- **USB drive with replug:** before every run the drive was **ejected, unplugged and plugged back in** (`bench --pause` waits until the drive's folder disappears and reappears), so nothing came from the file cache. One run per configuration (`results/task2.csv`, full output in `results/logs/2-usb.txt`), command below.
- **USB drive without replug:** the drive stayed plugged in, and the 8 files were dropped from the page cache before each run instead (`posix_fadvise(DONTNEED)`). This separates the start-up stall of the stick from the reading itself (`results/task2-usb-evicted.csv`).
- **Reference, internal NVMe SSD:** the same program on the SSD, 3 runs per configuration, with a cold cache (the 8 files dropped from the page cache before every run, 352 MB each time) and with a warm cache (the files already in RAM).

```
lab1 bench --task 2 --threads 1,2,4 --runs 1 --pause --dir /run/media/$USER/7CFE-8E4F/lab1 \
           --csv results/task2.csv --expect data/numbers.txt.counts
```

5 Experimental Results

5.1 Task 1a: Counting with Threads

Table 3 shows the median of 5 runs for each number of threads; the efficiency is E = S/p. The totals were correct in every run. Figures 1–3 show Task 1a and Task 1c together, so that they can be compared (question 5).

Table 3: Task 1a – time, speed-up, efficiency and peak memory vs. number of threads

Threads	Time, s	Speed-up	Efficiency	Peak memory, MiB	Totals correct
1	0.545	1.00×	100.0 %	31.9	yes (5/5)
2	0.277	1.97×	98.4 %	32.1	yes
4	0.150	3.64×	91.0 %	32.7	yes
8	0.098	5.54×	69.2 %	33.7	yes
16	0.079	6.93×	43.3 %	36.2	yes
32	0.080	6.83×	21.4 %	40.6	yes
64	0.076	7.16×	11.2 %	46.7	yes

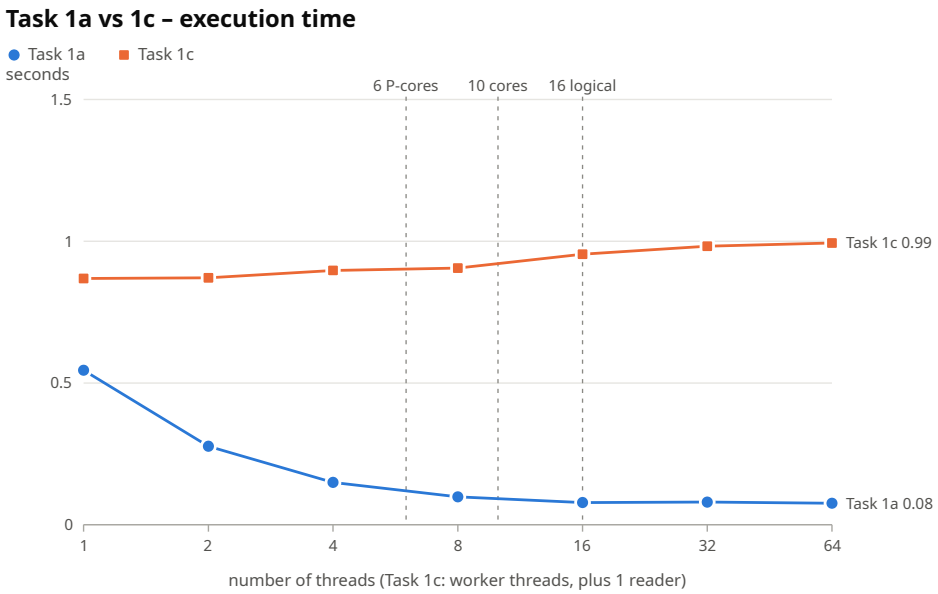


Figure 1: Task 1a and Task 1c – wall-clock time vs. number of threads

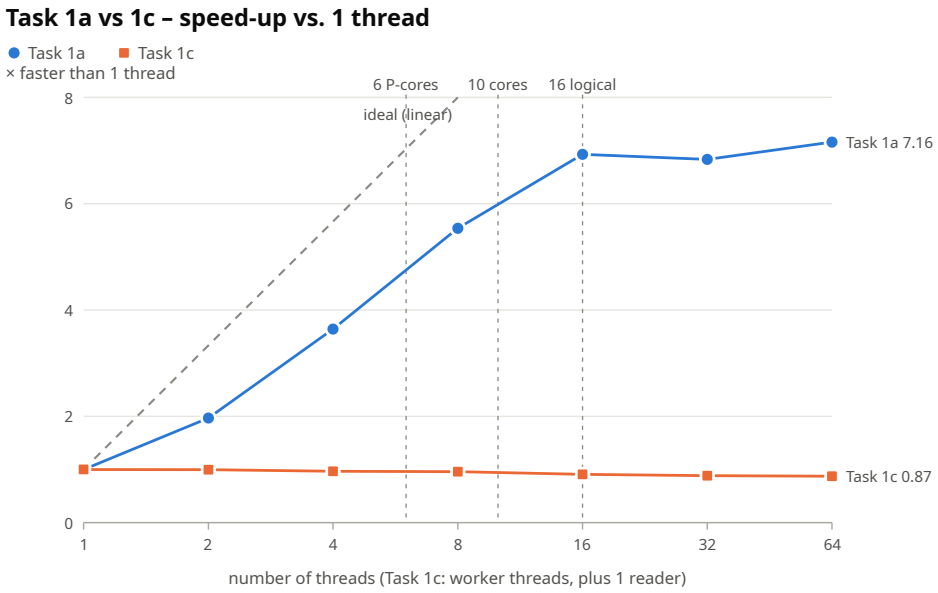


Figure 2: Task 1a and Task 1c – speed-up vs. number of threads

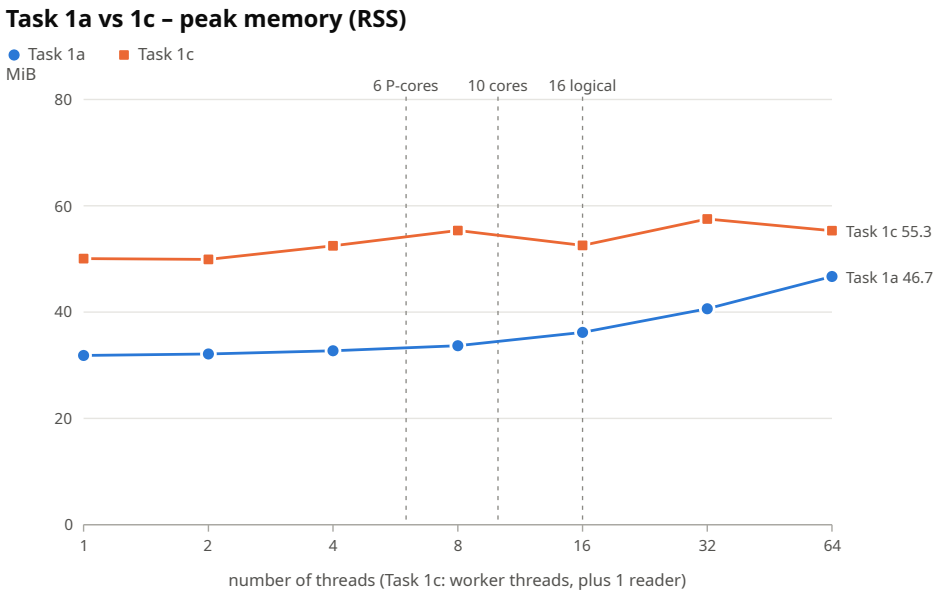


Figure 3: Task 1a and Task 1c – peak memory vs. number of threads

5.2 Task 1b: Three Ways to Count

Table 4 shows the main run (`Lab1 1b`, 16 threads, `results/logs/1b-main.txt`).

Table 4: Task 1b – main run with 16 threads

Version	Time, s	Negative	Positive	Zero	Total	Correct
1. shared, no synchronisation – run 1	0.142	7 465 496	7 333 006	25	14 798 527	**no**, 35 201 473 updates lost
1. shared, no synchronisation – run 2	0.178	6 705 746	6 830 429	25	13 536 200	**no**, 36 463 800 lost
1. shared, no synchronisation – run 3	0.169	7 459 187	7 384 942	25	14 844 154	**no**, 35 155 846 lost
2. shared + lock	3.476	25 000 383	24 999 592	25	50 000 000	yes
extra: shared + `Interlocked`	1.089	25 000 383	24 999 592	25	50 000 000	yes
3. per-thread counters	**0.020**	25 000 383	24 999 592	25	50 000 000	yes

A second, independent run (`results/task1b.csv`) gave the same picture: unsynchronised 0.146 / 0.127 / 0.445 s with 35.1 M / 35.8 M / 42.4 M lost updates, lock 3.687 s, `Interlocked` 1.145 s, per-thread 0.020 s.

The same four versions were also run with 1–64 threads (`results/task1b-sweep.csv`, median of 3 runs): Table 5 and Figures 4–6 (time on a logarithmic scale).

Table 5: Task 1b – time of the four versions vs. number of threads

Threads	unsync, s	lock, s	Interlocked, s	per-thread, s
1	0.192 (correct)	0.753	0.324	0.202
2	0.112 (wrong)	1.691	0.941	0.101
4	0.074 (wrong)	2.849	1.004	0.053
8	0.065 (wrong)	2.940	0.514	0.034
16	0.157 (wrong)	3.985	0.594	0.023
32	0.164 (wrong)	2.495	1.034	0.024
64	0.153 (wrong)	3.571	0.584	0.026

Peak memory is ≈415 MiB in every case: it is the 200 MB `int[]` plus the memory used while loading it, and does not depend on the version or the number of threads.

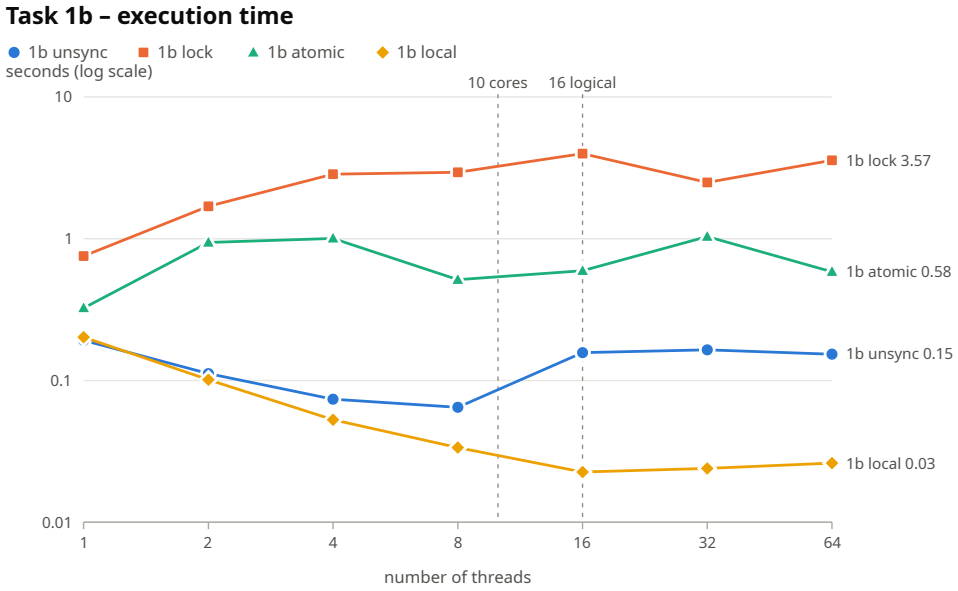


Figure 4: Task 1b – time of the four versions vs. number of threads (logarithmic scale)

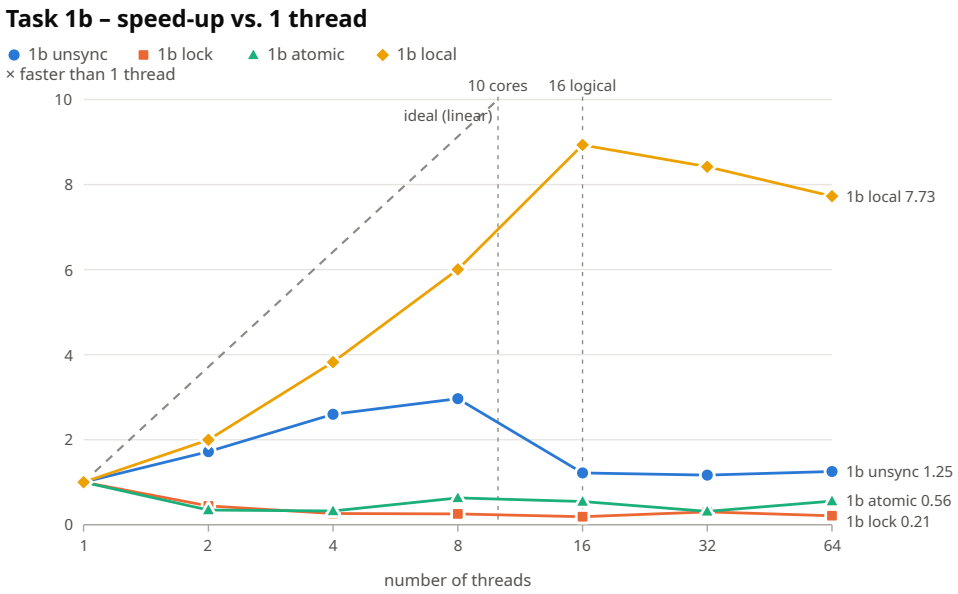


Figure 5: Task 1b – speed-up of the four versions vs. number of threads

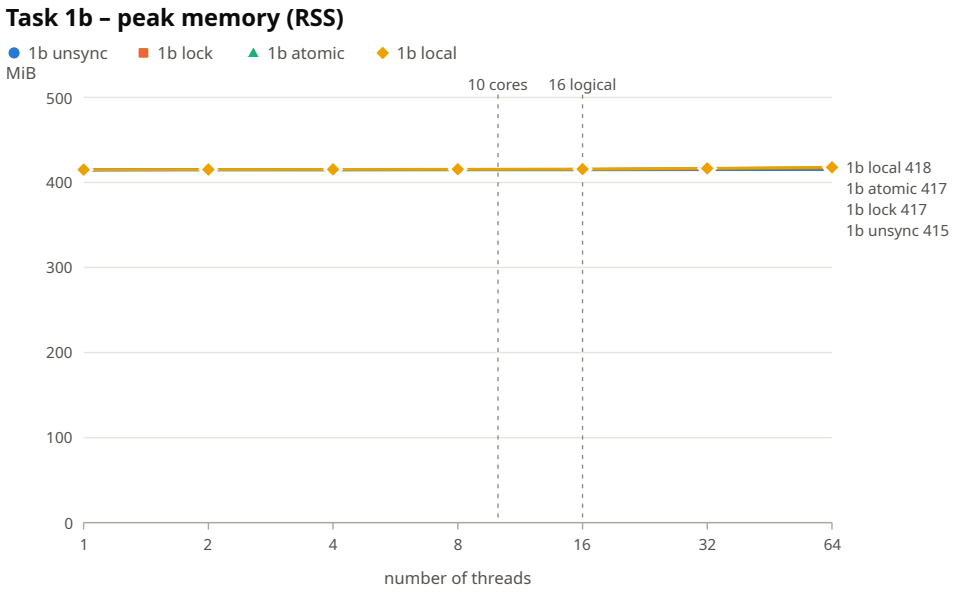


Figure 6: Task 1b – peak memory vs. number of threads

5.3 Task 1c: Producer and Consumers

Table 6 shows the median of 3 runs with batches of 10 000 lines and a queue capacity of 100. The charts of Task 1c are the orange series in Figures 1–3.

Table 6: Task 1c – time, peak memory and queue statistics vs. number of workers

Workers	Time, s	Speed-up	Peak memory, MiB	Reader waited on full queue	Longest queue	Avg. worker wait on empty queue	Correct
1	0.868	1.00×	50.1	0 times	3–4 batches	0.28–0.33 s	yes
2	0.871	1.00×	49.9	0	4–5	0.57–0.63 s	yes
4	0.897	0.97×	52.5	0	4–6	0.74–0.78 s	yes
8	0.905	0.96×	55.4	0	2–4	0.81–0.83 s	yes
16	0.954	0.91×	52.5	0	4	0.86–0.98 s	yes
32	0.983	0.88×	57.5	0	4–5	0.92–1.00 s	yes
64	0.994	0.87×	55.3	0	4–7	0.97–1.01 s	yes

Table 7 shows the extra experiments for question 6 (1 run each, `results/task1c-batch1*.csv`, `results/task1c-unbounded.csv`, table `results/charts/task1c-queue-table.md`).

Table 7: Task 1c – effect of the batch size and of the queue capacity

Configuration	Workers	Time, s	Peak memory, MiB	Longest queue
batch 1 line, capacity 100	1	17.03	44.3	100 (full 57 512 times)
batch 1 line, capacity 100	4	36.98	44.9	100
batch 1 line, capacity 100	16	40.31	45.0	100
batch 10 000, **unbounded**	1 / 4 / 16	0.852 / 0.893 / 0.937	50.2 / 55.3 / 53.9	4–6
batch 1 line, **unbounded**	1	17.52	49.5	**40 203**

5.4 Task 2: Many Files on Slow Storage

5.4.1 USB Drive with Replug

Table 8: Task 2 on the USB drive, drive replugged before every run

Threads	Time, s	Speed-up	Peak memory, MiB	Reading (thread-time sum), s	Parsing (sum), s	Overall MB/s	Correct
1	**58.78**	1.00×	77.8	58.27	0.51	6.3	yes
2	**59.62**	0.99×	121.8	118.64	0.51	6.2	yes
4	**59.18**	0.99×	386.1	235.89	0.52	6.2	yes

The per-file times (Table 9) show what happens inside a run.

Table 9: Task 2 on the USB drive – per-file read times

Threads	First file of each thread	Every later file	Read speed per thread → all threads together
1	34.7 s	3.3–3.5 s	13.4–14.1 MB/s → **≈13.6 MB/s**
2	38.2 s (both)	7.0–7.1 s	6.5–6.6 MB/s → **≈13.1 MB/s**
4	45.0 s (all four)	13.8–14.0 s	3.3 MB/s → **≈13.2 MB/s**

Every run starts with a **stall of ≈31 s** (for example 1 thread: 34.7 s − 46.2 MB / 13.6 MB/s = 31.3 s; 2 threads: 38.2 s − 92.4 MB / 13.3 MB/s = 31.2 s; 4 threads: 31.1 s), the same with any number of threads. The kernel log shows no errors, resets or timeouts. The stall happens only right after the stick is plugged in: a read started two minutes after plugging in ran at the full 13.6 MB/s from the first byte. It is most likely the stick's cheap controller starting up after power-on.

5.4.2 USB Drive without Replug

To separate the start-up stall from the reading itself, the three configurations were run once more while the drive stayed plugged in, with the 8 files dropped from the page cache before each run (the 13.6 MB/s read speed confirms that the data came from the stick, not from RAM). Results in Table 10 (`results/task2-usb-evicted.csv`); Figures 7–9 show both USB series next to the SSD reference of Section 5.4.4.

Table 10: Task 2 on the USB drive without replug (files dropped from the page cache)

Threads	Time, s	Speed-up	Reading (thread-time sum), s	Parsing (sum), s	Overall MB/s	Peak memory, MiB	Correct
1	**27.39**	1.00×	26.87	0.52	13.5	77.7	yes
2	**28.91**	0.95×	57.21	0.52	12.8	253.9	yes
4	**28.60**	0.96×	113.68	0.52	12.9	385.9	yes

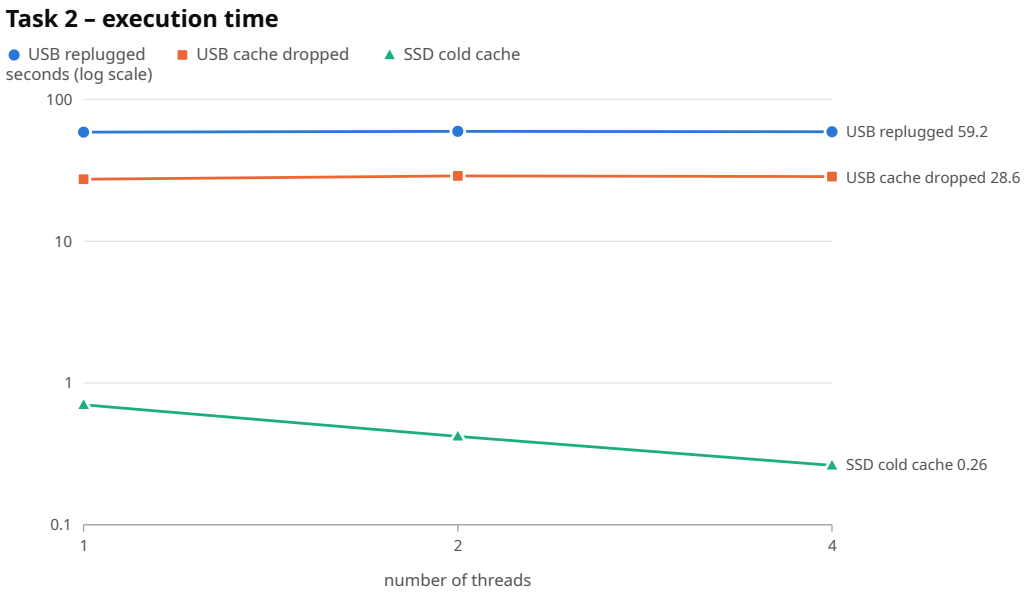


Figure 7: Task 2 – time on the USB drive (with and without replug) and on the SSD (logarithmic scale)

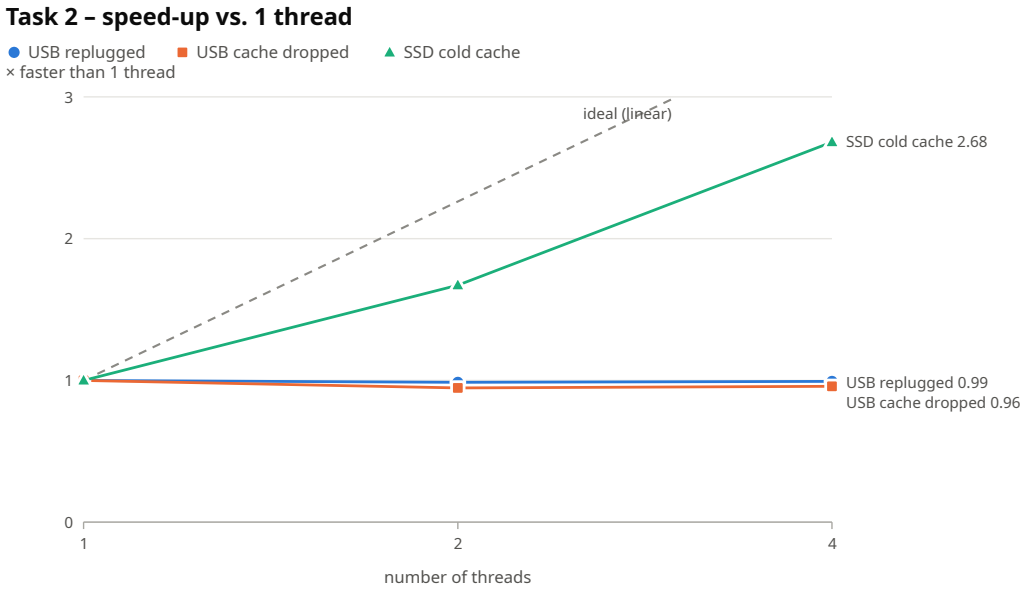


Figure 8: Task 2 – speed-up on the USB drive and on the SSD

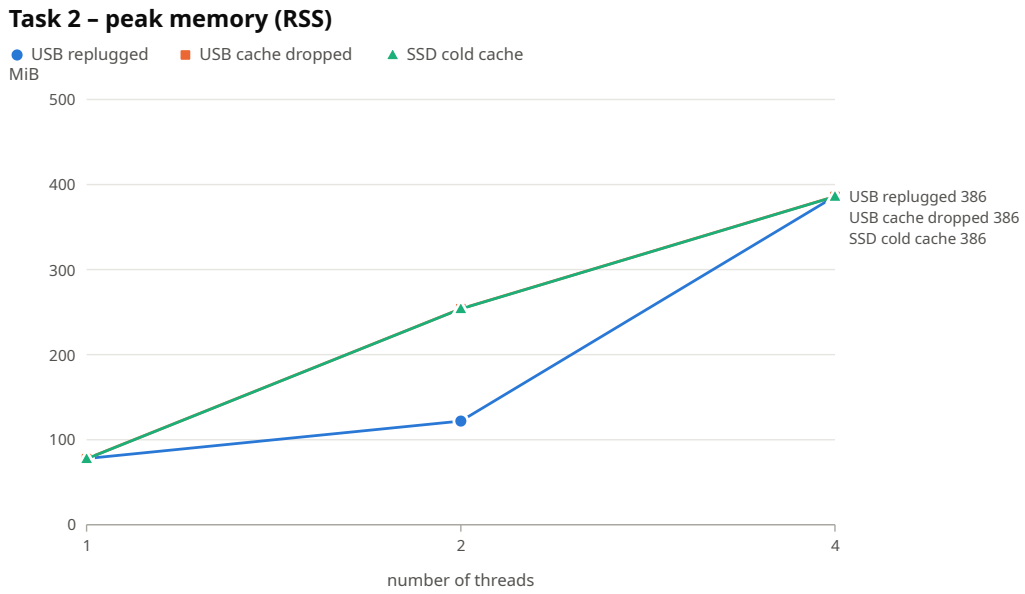


Figure 9: Task 2 – peak memory on the USB drive and on the SSD

5.4.3 A Note on the First Attempt

After copying the files, the stick was once pulled out without being ejected. The stick had not finished writing: the last 2.7 MB of `part_8.txt` and the whole of `expected.counts` came back as random bytes. The program refused the corrupt data. The two files were copied again, the drive was ejected properly, and all the runs above produced the correct totals. This is why the lab insists on *ejecting* before unplugging.

5.4.4 Reference: Internal NVMe SSD

Table 11: Task 2 on the internal NVMe SSD, cold and warm page cache

Threads	SSD cold, s	Speed-up	Reading (sum), ms	Parsing (sum), ms	SSD warm, s	Speed-up	Peak memory, MiB
1	0.702	1.00×	180–188	515–527	0.583	1.00×	77
2	0.420	1.67×	297–313	517–533	0.315	1.85×	122–254
4	0.262	2.68×	483–486	524–541	0.172	3.40×	386

On the SSD, one thread spends 0.18 s reading 369 MB (≈2 GB/s) and 0.52 s parsing it, so there the work is **CPU-bound** and 2 and 4 threads speed it up (2.7×), like Task 1a.

Peak memory grows with the threads, on the USB drive and on the SSD alike (77 → 122–254 → 386 MiB), because every thread holds a whole 46 MB file in memory while it parses it. A 46 MB array is a large object that the .NET GC frees only during a full collec-

tion, so a few finished buffers can still be resident when the next ones are allocated. That is why the 2-thread value varies between runs (122 or 254 MiB).

6 Task Questions

6.1 Task 1 Questions

6.1.1 Question 1: At which number of threads does the speed-up stop growing? Relate it to physical cores and logical processors.

At **16 threads = the number of logical processors**. Up to 4 threads the speed-up is almost ideal (1.97×, 3.64×), because each thread can run alone on a fast P-core. From 4 to 8 it grows more slowly (5.54×) and from 8 to 16 only by another 25 % (6.93×). The 6 P-cores are used up by the first 6 threads. The next threads either run on E-cores, which are slower (3.4 GHz instead of 4.6 GHz, simpler cores), or run as the second hardware thread of a P-core, sharing that core's execution units with the first thread. So the 16 logical processors are worth much less than 16 cores, and even 10 physical cores are not 10 equal cores. Beyond 16 threads there is no more hardware to run them on, and the speed-up stays flat at ≈7×. The per-thread version of Task 1b, which only reads an array, shows the same knee: 3.82× at 4, 6.00× at 8, **8.93× at 16**, then 8.42× and 7.73×.

6.1.2 Question 2: Why do 32 and 64 threads not help? Are they slower, and why?

They do not help because only 16 threads can run at the same moment; the other threads wait for a time slice and the same amount of CPU work is just cut into more pieces. On this machine they are **not measurably slower** for Task 1a (0.080 s and 0.076 s against 0.079 s; the five runs with 16 threads alone range from 0.072 s to 0.086 s). The extra costs exist (creating and joining 48 more threads, more context switches, more 256 KiB buffers and stacks, 64 boundary searches), but they are a few milliseconds out of 76 ms. They are partly cancelled by better load balancing: with 64 small parts, a thread stuck on a slow E-core holds up less work at the end (see Section 7.6). Where threads compete for a shared resource the extra threads do hurt: Task 1b per-thread counting goes from 0.023 s (16) to 0.026 s (64), and Task 1c gets slower with every extra worker (0.87 s → 0.99 s).

6.1.3 Question 3: Does the peak memory depend on the number of threads? Why?

Yes, but only a little and linearly: 31.9 MiB with 1 thread, 36.2 MiB with 16, 46.7 MiB with 64, i.e. ≈0.23 MiB per thread. Each thread has its own 256 KiB read buffer, its own stack (only the touched pages count) and runtime bookkeeping. The file itself is **never loaded as a whole**: every thread streams through its part, so memory does not depend on the file size. The ≈31 MiB base is the .NET runtime and the compiled code.

6.1.4 Question 4: Rank the three versions of Task 1b by time. Which of them are correct? Explain each result.

Fastest to slowest (16 threads): **per-thread counters 0.020 s < unsynchronised 0.13–0.45 s < (Interlocked 1.09 s) < lock 3.5–3.7 s**. Only the **per-thread** and the **lock** versions (and Interlocked) are correct; the unsynchronised version is wrong every time and by a different amount each time (35.2 M, 36.5 M, 35.2 M, 35.1 M, 35.8 M, 42.4 M of 50 M increments lost).

- *Unsynchronised*: `counter++` is three steps (load, add 1, store). When two threads load the same value, both store value + 1 and one increment disappears (lost update). It is also slow for a version without locks: the three counters are in one 64-byte cache line that 16 cores write to all the time, so the line keeps moving from core to core (cache-coherence traffic). With 16 threads it is even slower than with 8 (0.157 s vs 0.065 s). Only `zero` is right (25), because zeros are so rare that two threads never update it at the same moment.
- *Lock*: correct because the read-modify-write happens inside one critical section, but 50 000 000 lock/unlock pairs are executed and only one thread at a time can be inside. The threads effectively take turns, plus the lock itself is a contended shared cache line, and waiting threads may be put to sleep and woken up by the OS. Result: slower with 16 threads (3.5 s) than with **one** thread (0.75 s), and 170× slower than per-thread counters.
- *Interlocked* (extra): an atomic `lock xadd` instruction is correct and cheaper than a lock, but every increment still fights for the same cache line → 1.1 s.
- *Per-thread*: each thread counts in its own local variables (in CPU registers); nothing is shared while counting, and there is one write per thread at the end. No synchronisation, no cache-line traffic → it scales (8.9× at 16 threads) and is correct.

6.1.5 Question 5: Compare Task 1c with Task 1a. Which one is faster, and why? At which number of workers does Task 1c stop getting faster?

Task 1a is much faster: 0.079 s with 16 threads against 0.87 s for the best 1c configuration (**11× faster**); even 1a with a single thread (0.545 s) beats every 1c configuration. Task 1c

does not get faster at all: it is already at its best with 1 worker (0.868 s with 1, 0.871 s with 2, then slower). The measurements show why. The reader never once found the queue full and the queue never held more than 7 of the 100 allowed batches, while the workers spent most of their time waiting for an empty queue to be refilled (one worker: 0.3 s of 0.87 s; with 64 workers: almost the whole run). The **single reader thread is the bottleneck**. It does the sequential part of the work: reading, finding line ends, converting UTF-8 bytes to UTF-16 `string`s and allocating one object per line: one run of Task 1c allocates **2 269 MiB** of short-lived objects (50 M strings + the batch arrays), against **4 MiB** for the whole of Task 1a (`GC.GetTotalAllocatedBytes`). The parsing that the workers do in parallel is the cheap part. By Amdahl's law the time cannot fall below the reader's ≈0.85 s, and more workers only add contention on the queue and wake-ups. In Task 1a reading and splitting lines are also done in parallel, directly on bytes, without allocations.

6.1.6 Question 6: What would happen with batches of a single line? And with a queue that has no size limit?

Measured (Table 7). With **batches of one line** every number costs one `Add` and one `Take` on the shared queue (locking and signalling, often waking a sleeping thread through the OS): 50 000 000 queue operations instead of 5 000. One worker took **17.0 s (20× slower)**, and more workers made it worse (4 workers: 37.0 s, 16 workers: 40.3 s). The workers fight over the queue's lock and wait almost the whole time (31.8 s of 37 s with 4 workers). The synchronisation cost per item is larger than the work per item, so batching is what makes the queue usable.

With an **unbounded queue** and 10 000-line batches nothing changed (0.85–0.94 s, 50–55 MiB), because here the consumers are faster than the producer and the queue never holds more than 6 batches. The limit matters when the producer is faster than the consumers. Then an unbounded queue keeps growing and memory is not bounded any more: in the one configuration where the workers fell behind (batch 1, 1 worker) the queue grew to **40 203 entries** instead of stopping at 100. With 10 000-line batches and slow workers, the whole file (≈50 M strings, gigabytes) could end up in the queue, up to an out-of-memory error. A bounded queue gives **back-pressure**: the reader is made to wait, and the memory stays below 100 × 10 000 lines (≈40 MB of strings).

6.2 Task 2 Questions

6.2.1 Question 1: Compare the time on one thread with Task 1a on one thread. On the USB drive, is the work CPU-bound or I/O-bound?

I/O-bound. With one thread Task 2 took **58.8 s** on the USB drive, against **0.545 s** for Task 1a and 0.70 s for the same Task 2 on the SSD: **108× slower** for exactly the same numbers and the same parsing code. Of the 58.8 s, **58.3 s (99 %) were spent waiting in `ReadAllBytes`** and only **0.51 s (0.9 %) parsing**. The parsing time is the same as on the SSD (0.52 s) and in Task 1a, so the CPU did not do more work; it just waited. Even without the ≈31 s start-up stall (drive left plugged in, cache dropped) it is 27.4 s, of which 26.9 s (98 %) is reading. The throughput is **13.5 MB/s**, the limit of this USB 2.0 stick, while one core can parse about 700 MB/s (369 MB in 0.52 s). The CPU was idle more than 98 % of the time.

6.2.2 Question 2: On the USB drive, do more threads help? Why not, when lecture 1 showed that I/O-bound work benefits from concurrency?

No. 1 / 2 / 4 threads: 58.8 / 59.6 / 59.2 s with the replug (speed-up 1.00 / 0.99 / 0.99), and 27.4 / 28.9 / 28.6 s without it (1.00 / 0.95 / 0.96, so slightly *slower*). The per-file times show why. Every thread waits for **the same single device**, and its bandwidth is fixed at ≈13.5 MB/s. One thread reading one file sequentially already uses all of it. With 2 threads each gets half (6.5 MB/s), with 4 threads a quarter (3.3 MB/s): the total stays ≈13 MB/s, so the total time stays the same. It is even a little worse, because the stick has to jump between 2 or 4 files instead of streaming one, and the only work that can overlap, parsing (0.5 s in total), is too small to matter. The start-up stall (≈31 s) also cannot be hidden by more threads: all of them wait for the same stick.

Concurrency helps I/O-bound work when the waits are **independent latencies** (e.g. many network requests, each waiting for a different remote server, while the link and the CPU are idle): then many waits overlap. It does not help when all threads queue for **one resource that is already at its throughput limit**, which is exactly what a USB 2.0 stick is.

7 Control Questions

7.1 What is the difference between a physical core and a logical processor? How did it show in your charts?

A physical core is real execution hardware: its own arithmetic units, registers, and L1/L2 caches. A logical processor is what the OS schedules threads on. With Hyper-Threading (SMT) one P-core appears as two logical processors. They have separate register sets, but they share the core's execution units and caches, so two threads on one core run only ≈20–30 % faster than one, not 2×. This CPU has 6 P-cores × 2 + 4 E-cores × 1 = 16 logical processors on 10 physical cores, and the E-cores are slower still. In the charts (Figure 2): Task 1a scales almost linearly up to 4 threads, flattens between 8 and 16 (5.5× → 6.9×, far from 16×), and stops at 16, the number of logical processors.

7.2 How did you make sure that no line was counted twice or skipped at the boundary between two parts of the file?

Each thread gets a raw byte range [size·i/N, size·(i+1)/N). Rule: *a line belongs to the thread whose range contains its first byte.* Every thread moves **both** ends of its range with the same function, `AlignToLineStart(offset)`: if the byte before `offset` is `\n`, then `offset` is already a line start; otherwise move forward to the byte after the next `\n`. Thread i's aligned end is computed exactly like thread i+1's aligned start, so the aligned ranges fit together with no gap and no overlap, and each thread parses only complete lines inside its range. Checks: `--verbose` prints the ranges and confirms they tile the file; every run of every thread count gives exactly 50 000 000 and the same totals as the generator; edge cases were tested (more threads than lines, last line without `\n`, Windows `\r\n` line endings).

7.3 Explain step by step how unsynchronised shared counters lose updates.

`negative++` is executed as: (1) load the value from memory into a register, (2) add 1, (3) store the register back. Even a single `inc [mem]` instruction does these three steps without the `lock` prefix and is not atomic across cores. Example with `negative = 100`:

1. thread A loads 100;
2. thread B loads 100;
3. A computes 101 and stores 101;
4. B computes 101 and stores 101.

Two numbers were counted but the counter grew by one, and B's store overwrote A's update. With 16 threads hitting the same counters millions of times, this happened to 70–85 % of the increments, and differently in each run.

7.4 Why are per-thread counters faster than shared counters protected by a lock?

With a lock every one of the 50 M increments needs an acquire and a release (atomic instructions on the lock's memory). Only one thread at a time can hold the lock, so the threads run one after another instead of in parallel. The lock and the counters move between the cores' caches on every operation, and a thread that finds the lock taken may be put to sleep and woken up by the OS. Per-thread counters share nothing: they stay in registers, need no atomic instruction and no cache-line transfer, and are combined once after `Join`. Measured: 0.020 s vs 3.5 s (≈170×).

7.5 What does "I/O-bound" mean for a USB drive, and for a server waiting on the network? Why does concurrency help in only one of the two cases?

I/O-bound means the program spends most of its time waiting for input/output, not computing.

- For the **USB drive** the limit is the **bandwidth of one device**: a single sequential reader already keeps it 100 % busy, so more threads only share the same MB/s (and may make access less sequential). Measured: one thread read at 13.6 MB/s; with 2 threads each got 6.5 MB/s, with 4 threads each got 3.3 MB/s, and the total time did not change.
- For a **server waiting on the network** the time goes on **latency**: each request waits, e.g., 100 ms for a remote machine to answer, while the network link and the CPU are mostly idle. Those waits are independent of each other, so many requests can wait at the same time and 100 concurrent requests take about as long as one.

Concurrency helps when it overlaps waiting on *independent* resources, and does not help when the threads queue for the *same* saturated resource.

7.6 What is the difference between dividing work into equal parts in advance and taking it from a queue? When is each one better?

- *Static partitioning* (Task 1a, Task 2): each thread knows its part from the start. There is no coordination during the work, no shared state, no allocations, and good locality, but

the total time is the time of the **slowest** part. It is best when the work splits evenly, its size is known in advance (a file you can seek in, an array), and the cores are equal.

- *A shared queue* (Task 1c): threads take the next piece when they are free, which **balances the load automatically**. In 1c some workers took more than 4× as many batches as others (e.g. 46 vs 206 batches with 64 workers), which is what you need on a hybrid P/E CPU or when items have different costs. It also works for a stream whose size is unknown and that can only be read in order. The price is synchronisation on every take, so items must be batched (batch = 1 was 20× slower), and a single producer can become the bottleneck (it was in 1c).

The queue is better for uneven or unknown work, streaming input or uneven cores; static parts are better for uniform, splittable work.

8 Conclusions

1. **CPU-bound work and the hardware limit.** Task 1a, Task 1b and Task 1c, and Task 2 on the internal SSD, are CPU-bound. Evidence: (1) the time falls almost in proportion to the number of threads until the cores run out (Task 1a: 3.6× at 4 threads, 6.9× at 16; per-thread counting in 1b: 8.9×), and stops exactly at the hardware limit of 16 logical processors; (2) reading the whole data set from the SSD takes only 0.18 s of the 0.70 s single-thread time of Task 2, the rest is parsing.
2. **The sequential part limits the speed-up.** Task 1c is CPU-bound too, but on **one** thread (the reader), so adding workers cannot help. This is Amdahl's law: the sequential part limits the speed-up.
3. **I/O-bound work.** Task 2 on the USB drive is I/O-bound. Evidence: 99 % of the time (58.3 s of 58.8 s) is spent waiting for `ReadAllBytes`, and the parsing takes the same 0.5 s as everywhere else; the throughput (13.5 MB/s) is the stick's limit, 50× below what one core can parse; and more threads change nothing (0.95–0.99×) because they all share the same device bandwidth. The same code on the SSD is CPU-bound and speeds up 2.7× with 4 threads. Whether a task is CPU- or I/O-bound is not a property of the code, but of the code on a particular piece of hardware.
4. **Synchronisation cost.** Sharing a counter is either wrong (unsynchronised: 70–85 % of the updates lost) or very slow (lock: 3.5 s, 170× slower than per-thread counters, slower than one thread). The fast and correct design shares nothing while working and combines the results once at the end.

9 Source Code and Build Instructions

The source code is in the `Lab1` folder of the repository https://github.com/m0sji/PR_26; `README.md` describes every command in detail. The program requires the .NET 10 SDK.

9.1 Build

```
cd Lab1
dotnet build -c Release
alias lab1='dotnet bin/Release/net10.0/Lab1.dll'   # used below
lab1 help
```

9.2 Generating the Data

```
lab1 gen      # data/numbers.txt: 50 000 000 integers (seed 2026) + numbers.txt.counts
lab1 split    # data/parts/part_1.txt .. part_8.txt (6 250 000 lines each) + expected.counts
```

9.3 Running One Configuration

```
lab1 1a --threads 8                # Task 1a (--verbose shows each thread's byte range)
lab1 1b                            # Task 1b, threads = logical CPUs, unsync run 3 times
lab1 1b --mode lock --threads 4    #   one version only: unsync | lock | atomic | local
lab1 1c --threads 8                # Task 1c: 8 workers + 1 reader, batch 10 000, capacity 100
lab1 1c --threads 8 --batch 1      #   batches of one line
lab1 1c --threads 8 --capacity 0   #   unbounded queue
lab1 2 --dir /media/$USER/USB/lab1 --threads 2     # Task 2
```

Each run prints the time, the peak memory, the totals and a machine-readable `@result` line.

9.4 Reproducing the Measurements

```
lab1 bench --task 1a --runs 5 --csv results/task1a.csv
lab1 bench --task 1b --threads 16 --runs 1 --warmup 0 --csv results/task1b.csv
lab1 bench --task 1b --threads 1,2,4,8,16,32,64 --runs 3 --warmup 0 --repeat 1 \
           --csv results/task1b-sweep.csv
lab1 bench --task 1c --runs 3 --csv results/task1c.csv
lab1 bench --task 1c --threads 1,4,16 --runs 1 --batch 1    --csv results/task1c-batch1.csv
lab1 bench --task 1c --threads 1,4,16 --runs 3 --capacity 0 --csv results/task1c-unbounded.csv
lab1 bench --task 2 --threads 1,2,4 --runs 1 --pause --dir /media/$USER/USB/lab1 \
           --csv results/task2.csv
```

`bench` starts a new child process for every run, so the peak memory belongs to that one configuration. It does one unrecorded warm-up run first, so that the file is in the OS page cache and every recorded run reads it from memory, not from the SSD. The charts are produced with `lab1 chart` (see `README.md`).